

Day 61 — `pip`, Packages, and Virtual Environments

You completed your first **60 days of Python**. Today you'll learn how Python projects install external packages and how to keep those packages isolated from other projects.

1. Day Number

Day 61

2. Topic

`pip`, packages, and virtual environments

3. Connection to What You Already Know

Until now, you have mostly worked with Python itself and its built-in features.

Real Python projects often use code written by other developers. For example, instead of writing everything yourself, you might install a package that helps with:

- making HTTP requests
- working with dates
- creating charts
- building websites

Today you'll learn the basic developer workflow for installing those packages safely.

4. Important Topics

Package

A **package** is reusable Python code that you can install and use in your own program.

For example:

```
import requests
```

Here, `requests` is a popular external Python package.

Library

A **library** is a collection of useful code made for a particular purpose.

In beginner conversations, people often use the words **package** and **library** almost interchangeably.

You can think of it like this:

| A library gives you useful tools.

| A package is one way those tools are distributed and installed.

pip

pip is Python's package installer.

It lets you install packages from the command line.

Example:

```
python -m pip install requests
```

This tells Python:

| Use pip to install the package named requests.

Installing Packages

A package can usually be installed with:

```
python -m pip install package_name
```

For example:

```
python -m pip install requests
```

After installation, your Python program can use it:

```
import requests
```

venv

venv is Python's built-in tool for creating a **virtual environment**.

Example:

```
python -m venv .venv
```

This creates a virtual environment in a folder named:

```
.venv
```

Activate

After creating the environment, you normally **activate** it.

Activation tells your terminal:

| For now, use the Python and installed packages from this virtual environment.

On Windows:

```
.venv\Scripts\activate
```

On macOS/Linux:

```
source .venv/bin/activate
```

You may then see something like this at the beginning of your terminal:

```
(.venv)
```

That is a useful sign that the virtual environment is active.

Deactivate

When you're finished working inside the environment:

```
deactivate
```

This returns your terminal to its normal Python environment.

5. Foundational Notes

Imagine you have two Python projects.

```
weather_project  
game_project
```

Your weather project may need one version of a package.

Your game project may need another version.

If every package is installed globally on your computer, the projects can interfere with each other.

A virtual environment gives each project its **own package space**.

Conceptually:

```
weather_project/  
  .venv/  
  weather.py  
  
game_project/  
  .venv/  
  game.py
```

The two `.venv` folders are separate.

Packages installed in one do not normally affect the other.

6. Why Virtual Environments Are Useful

Suppose Project A needs:

```
somepackage version 1
```

but Project B needs:

```
somepackage version 2
```

Without separate environments, managing those requirements can become messy.

With virtual environments:

```
Project A
└─ its own environment
    └─ package version 1
```

and:

```
Project B
└─ its own environment
    └─ package version 2
```

This makes projects easier to manage and reduces package conflicts.

A simple beginner rule is:

| One Python project → one virtual environment.

7. Easy Example

Suppose you create a folder:

```
day61_project
```

Move into it:

```
cd day61_project
```

Create a virtual environment:

```
python -m venv .venv
```

Activate it.

Windows:

```
.venv\Scripts\activate
```

macOS/Linux:

```
source .venv/bin/activate
```

Then install a small package such as requests:

```
python -m pip install requests
```

Check whether it is installed:

```
python -m pip show requests
```

You should see information such as:

```
Name: requests  
Version: ...  
Location: ...
```

The exact version number may be different on your computer.

When finished:

```
deactivate
```

8. Day 61 Problem Statement

Create a small Python project folder.

Inside that project:

1. Create a virtual environment.
2. Activate the virtual environment.
3. Install one simple package.
4. Verify that the package is installed.

Use:

```
requests
```

as the package for this exercise.

You do **not** need to build a large Python program today.

The goal is simply to understand the package-installation workflow.

9. Concepts Used

You will practice:

- project folders
 - terminal/command line
 - Python packages
 - pip
 - `python -m pip`
 - venv
 - virtual environment creation
 - activation
 - package installation
 - checking installed packages
 - deactivation
-

10. Thought Process

When starting a Python project, think:

```
Where is my project?
  ↓
Do I have a virtual environment?
  ↓
No → create one
  ↓
Activate it
  ↓
Install the package I need
  ↓
Verify the installation
  ↓
Work on the project
  ↓
Deactivate when finished
```

The important idea is:

```
create → activate → install → verify → work → deactivate
```

That sequence is worth remembering.

11. Beginner-Friendly Command-Line Pseudocode

Think of the steps like this:

```
CREATE a project folder  
MOVE into the project folder  
CREATE a virtual environment  
ACTIVATE the virtual environment  
INSTALL requests  
CHECK that requests exists  
DEACTIVATE the environment
```

You don't need to memorize every command immediately.

Focus first on understanding **why each step exists**.

12. Suggested Solving Approach

Use this simple developer workflow:

```
Project folder  
↓  
Virtual environment  
↓  
Activate  
↓  
Install package  
↓  
Verify package  
↓  
Start coding
```

For today's exercise, stop after verifying the package.

The main lesson is not requests itself.

The lesson is learning how developers prepare a clean Python environment before working on a project.

13. Easy Edge Cases

Package already installed

You may run:

```
python -m pip install requests
```

and see something similar to:

```
Requirement already satisfied
```

That is not an error.

It simply means the package is already installed in the Python environment you are currently using.

Virtual environment is not activated

Suppose you create `.venv` but forget to activate it.

Then:

```
python -m pip install requests
```

may install the package somewhere outside your project's virtual environment.

Before installing, look for something similar to:

```
(.venv)
```

in your terminal.

For example:

```
(.venv) C:\projects\day61_project>
```

That is a useful visual clue that your environment is active.

14. Expected Commands and Expected Result

A typical session might look like this.

Create the folder:

```
mkdir day61_project
```

Enter it:

```
cd day61_project
```

Create the environment:

```
python -m venv .venv
```

Activate it.

Windows:

```
.venv\Scripts\activate
```

macOS/Linux:

```
source .venv/bin/activate
```

Install the package:

```
python -m pip install requests
```

Verify it:

```
python -m pip show requests
```

Expected result:

```
Name: requests
Version: ...
Summary: ...
Location: ...
```

The exact text and version can differ.

Finally:

```
deactivate
```

After deactivation, the (.venv) marker should disappear from your terminal.

15. Hint Only

For the exercise, remember this sequence:

```
folder
↓
python -m venv .venv
↓
activate
↓
python -m pip install ...
↓
python -m pip show ...
```

↓
deactivate

If you get stuck, first ask yourself:

| Is my virtual environment actually activated?

That one check solves many beginner problems.

Day 61 takeaway

pip → installs external Python packages

venv → gives a project its own isolated Python environment

The core workflow to remember is:

Create environment → Activate → Install → Verify → Deactivate